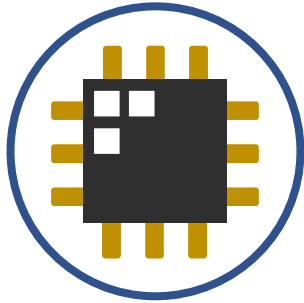




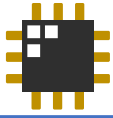
EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 3 – Memories: Flipflops/Registers

1

For week 11, we will be discussing flipflops and registers, and how they lay the foundation for processor designs. They are memory elements that allow for the concept of states allowing a digital circuit to not only depend on the current input but also the previous outputs or states as well. This enables increased complexity making a design of a processor possible.



Implementing more complex functions

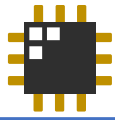
- Given what you know so far, implement this in hardware:

```
s = 0  
for i in range(9) :  
    s = s + 1
```

2

To motivate the need for memories, let's start with this Python code. This is simply a loop, incrementing the value of *s* by 1 9 times. *s* started with a value of 0, so by the end of the loop execution, we should end up with a value of *s* = 9. Pretty straightforward.

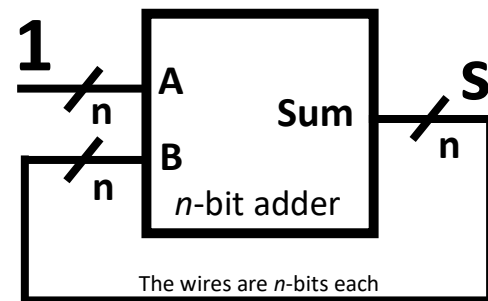
Now, if I ask you to implement this exact functionality using the knowledge you have so far. What would you do?



Implementing more complex functions

- Given what you know so far, implement this in hardware:

```
s = 0
for i in range(9):
    s = s + 1
```

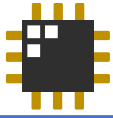


?

3

Well, you see the addition operation there; you can also see s being incremented using the previous value of s and then plus 1. So maybe, the thing that you are thinking about would resemble something like the figure on the right. Pretty straightforward: we are using an adder to perform the addition; we are using s from the output and loop it back to the input to be incremented by 1.

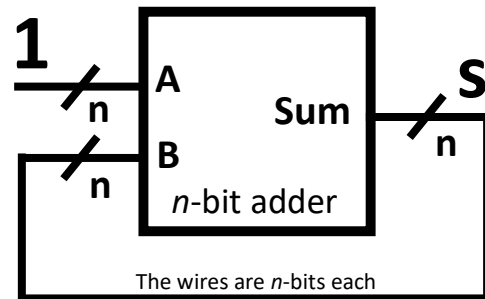
This logic circuit, however, will not work.



Implementing more complex functions

- Given what you know so far, implement this in hardware:

```
s = 0
for i in range(9):
    s = s + 1
```

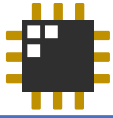


Issue 1: How do you set $s = 0$ at the beginning?

Issue 2: How do you control this? You only need to do it 9 times!

4

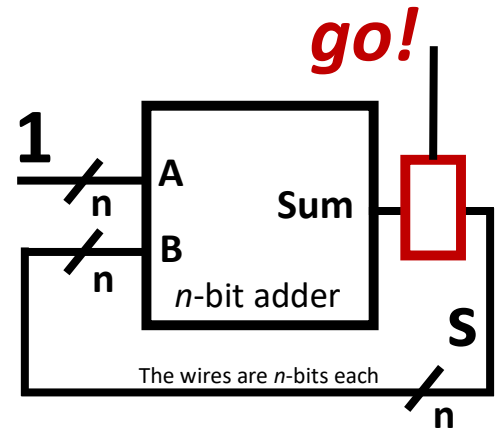
There are two issues on this circuit right now. The first is that we really cannot set s to be initially 0. There's no way in that circuit to force it. Now, we would technically solve it by putting an AND logic at the output, forcing s to be 0 (since anything ANDed to 0 is just 0). So, we have an additional input that becomes 0 to be ANDed to s , forcing the latter to be 0, and then becomes 1 to start the addition loop for s . That might solve the problem indeed. However, we also have another problem which is impossible to solve given the circuit right now. The Python code only wants to loop 9 times. There's no way in the circuit to perform the addition loop 9 times. We have no way to control the number of additions that will be performed. Control is the keyword here. For us to implement the code, we have to find a way to control the iterations and updates.



Implementing more complex functions

- We need a logic element that can:
 - **store** the current value of the output
 - be **triggered** for controlled updates
 - be **initialized** to some defined value

```
s = 0
for i in range(9):
    s = s + 1
```

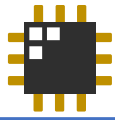


**trigger the go! signal
9 times and you're done**

To formalize the requirements, we need:

- 1) logic element that can hold the current value of the output, so that when the output is being used as the input for the next addition, the current value is held first, waiting for...
- 2) a trigger that will commence the 1-time update of the output. For every 'tick' of this trigger, the output would get the new value from the addition. Then we can just 'tick' this trigger 9 times to have the total iteration count to be also 9, consistent with the code that we had in the previous slide.
- 3) Finally, we also want to have a way to set the initial value of this logic element to 0, so that we know that the first addition that will be performed is $0 + 1$, just like what the Python code earlier wants.

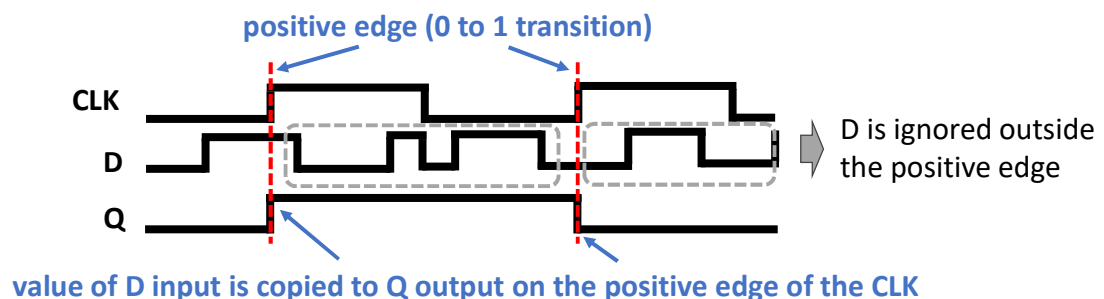
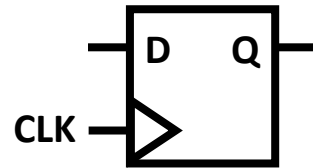
So far, all of the logic elements we have covered before cannot meet these requirements. We have to introduce a new one.



Flipflops

- A logic element that **stores** state

- Holds a 0 or 1 at the output **Q** independent of the **D** input and is **triggered** to update at every rising/positive edge of the clock (**CLK**) input.
- Has a reset input (not shown right now) allowing the output **Q** to be **initialized** to 0.



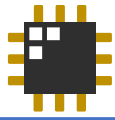
Let's introduce this new logic element. It is called a flipflop.

A flipflop is a logic element that stores a state, which can be 0 or 1. It is an edge triggered element that only updates its output or its state on one of the clock edges, typically a rising edge or positive edge of the clock.

A clock is another input signal to the flipflop, it basically just alternates between 0 and 1. You can imagine it like a blinking light, alternating off and on in a periodic fashion.

Now, the rising edge, also called as positive edge, is defined as the 0 to 1 transition of this clock signal. It's a single instant of time. It is during this instant of time when the flipflop captures whatever the input is and holds it.

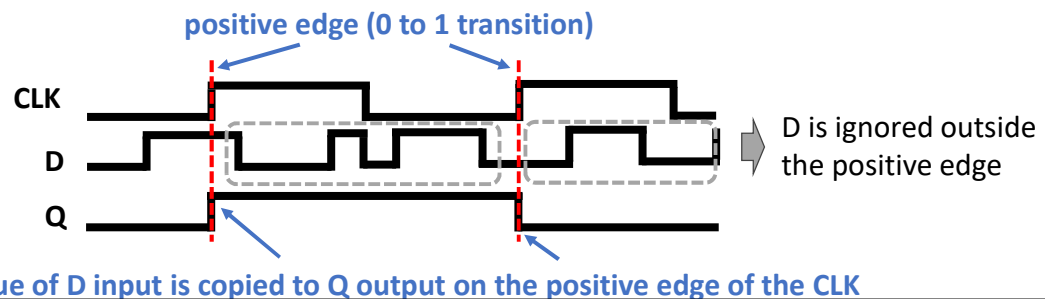
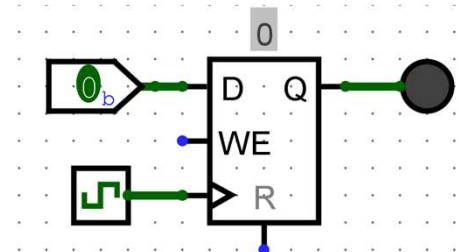
After the flipflop captures that value, you can technically change the input as many times as you want, and the flipflop will not respond to it. It will only capture the input again at the next positive edge of the clock. The flipflops only respond on the positive edge of the clock, it is only during this edge when the output of the flipflop changes. Remember this.



Flipflops

- A logic element that **stores** state

- Holds a 0 or 1 at the output **Q** independent of the **D** input and is **triggered** to update at every rising/positive edge of the clock (**CLK**) input.
- Has a reset input (not shown right now) allowing the output **Q** to be **initialized** to 0.

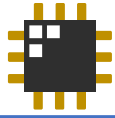


Here's a simulation of the flipflop. There are more pins in there as originally described, we can ignore them for now. Just for completeness:

WE signal means write enable, it is 1 by default, which is the normal flipflop behavior. If it is 0, it prevents the update of the output every positive edge trigger. We will use WE later in the lecture.

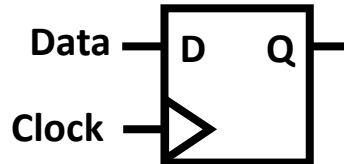
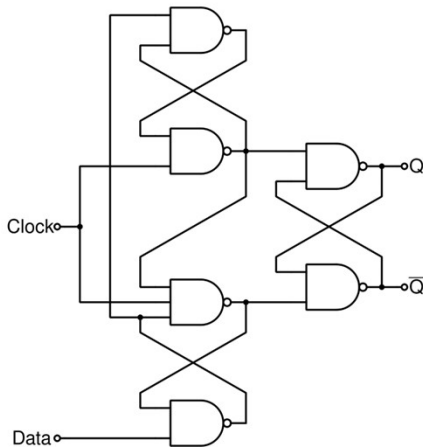
R signal means reset, forcing the output to be 0. This enables initialization for this logic element. At least for this simulation, the flipflop has already been initialized to 0.

In the simulation, we can see that, indeed, the output of the flipflop only changes during the positive edge of the clock. It doesn't matter if the input of the flipflop changes outside this positive edge event. The output will just copy whatever the input is on that specific time instant and then holds that value until the next positive edge of the clock happens again.



Flipflops

- Aside: Flipflops can be implemented using logic gates
 - Incorporating some sort of feedback enables them to **hold** values

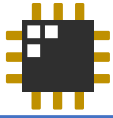


Analyzing what happens here for every change in **D (Data)** and **Clock** is a little tedious...

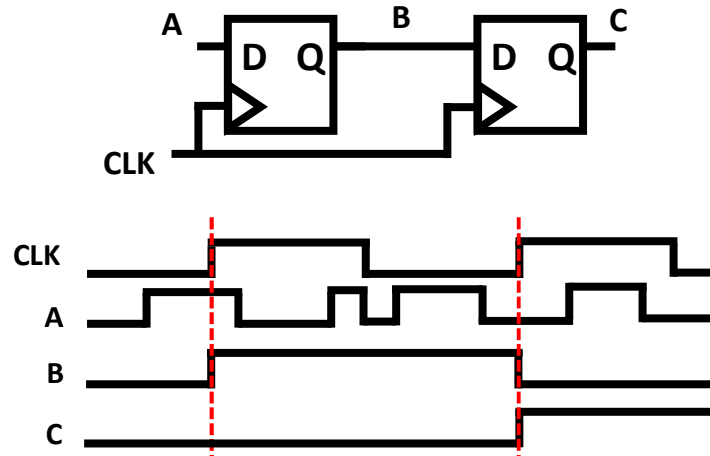
We can just abstract the functionality and approach it on a higher level.

More in-depth flipflop functionality in EEE 143!

Flipflops are actually implemented as logic gates as well, albeit a little more complicated. We will not discuss the logic gate level equivalent of flipflops as we will cover the flipflop functionality on a higher level. We don't have to analyze what is happening here in this diagram. It's mainly an illustration showing that flipflops are still implemented using the logic gates you already know. The logic gate being used there, by the way, is a NAND (NOT AND) gate, following the expression: $(AB)'$. More in-depth analysis on how flipflops work will be covered in EEE 143, so watch out for that!



Flipflop behavior

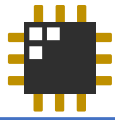


The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger. 9

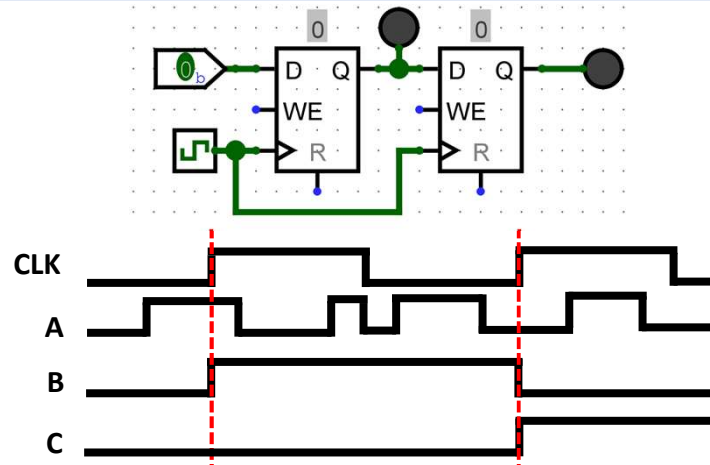
Let's explore what happens if we chain flipflops together. As you can see, both flipflops share the same clock, so they can both see the positive edge trigger event for them to update their values. The waveforms below show what exactly happens. Note how the output C (which is the output of the second flipflop on the right) is like a 1-cycle-delayed version of B (which is the output of the first flipflop on the left). The main thing to know why this is happening is that the positive edge of the clock is just a single instant in time. Q copying D from the input A (the first flipflop) will not propagate all the way to C in a single positive edge. When B copies A on the positive edge, C copies B at the same time, but not at the same value on that same positive edge. The flipflops both check the input values right before the positive edge and update right after the positive edge. Always!

You can also imagine the following description:

The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.
Always remember this flipflop description!



Flipflop behavior

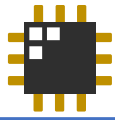


The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger. 10

Here's the corresponding demonstration of the two flipflops in series. We recall the following the statements:

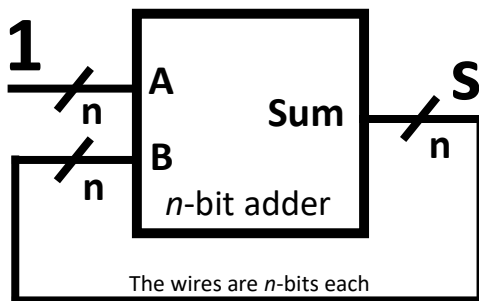
The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.

Indeed, when the first flipflop, on the left, captures the input during the positive edge, it was really checking the value of that input just right before the positive edge and then the output changes right after the positive edge. Therefore, the second flipflop on the right will see the value of the input (which is coming from the first flipflop) right before the change occurred. This is why the second flipflop acts as if it is delayed. The positive edge event is just a single instant in time. The outputs of the flipflops change just right after the positive edge of the clock has already passed.

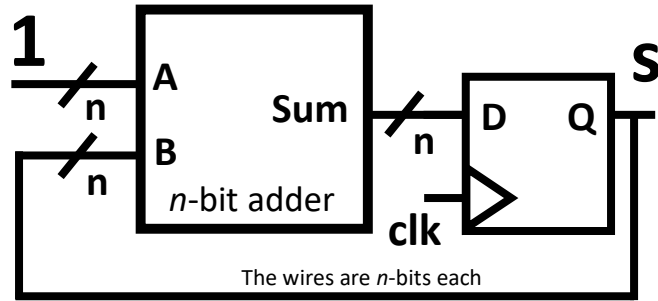


Accumulator design: Round 2

- Redesigning the accumulator



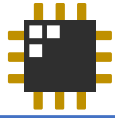
Can't be controlled!



Positive edge trigger for control!

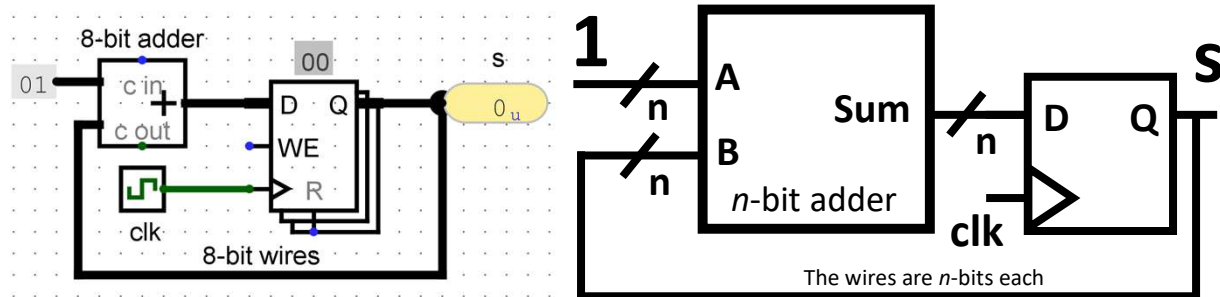
11

And thus, when trying to fix the accumulator solution that we had earlier, we can insert a flipflop at the output of the adder. The flipflop would hold the current value of *s*, then that same value is looped back to the adder to perform + 1. The adder output (which is now 1, since $0 + 1 = 1$) would go to the flipflop input. However, without the positive edge of the clock, the output *s* remains 0. Once we performed the positive edge of the clock, *s* now becomes 1. This updated value of *s* will propagate again back to the adder to perform + 1. The output of the adder (which is now 2, since $1 + 1 = 2$), will go to the flipflop input. However, since the positive edge of the clock has already passed, that new value will not go through *s*, *s* remains 1. It will need to wait for another positive edge of the clock to perform the update once again. This story continues for every positive edge of the clock. As you can see, we are controlling the output change using a trigger. This allows us to control how much addition is being performed. If we performed 9 positive edges of the clock, then the sum is updated 9 times. Compare that to the original solution we had earlier (shown on the left) and we'll see how that solution is technically an infinite loop without control, while the new version has the control (the positive edge of the clock) that we needed to implement the original Python code shown in the first few slides.



Accumulator design: Round 2

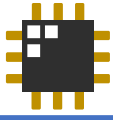
- Redesigning the accumulator



Positive edge trigger for control!

12

Here's a corresponding demonstration of the accumulator. Indeed, it needs to wait for the positive edge of the clock for s to update. For every positive edge of the clock, we see s updating to $s + 1$, due to the connected adder. The updates are always controlled by the positive edge event. To replicate the Python code we saw earlier, we have to trigger the positive edge event 9 times.

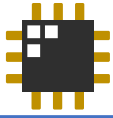


Introducing more complexity

- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)
 - We need to add some logic. *Hint*: we covered this last meeting!

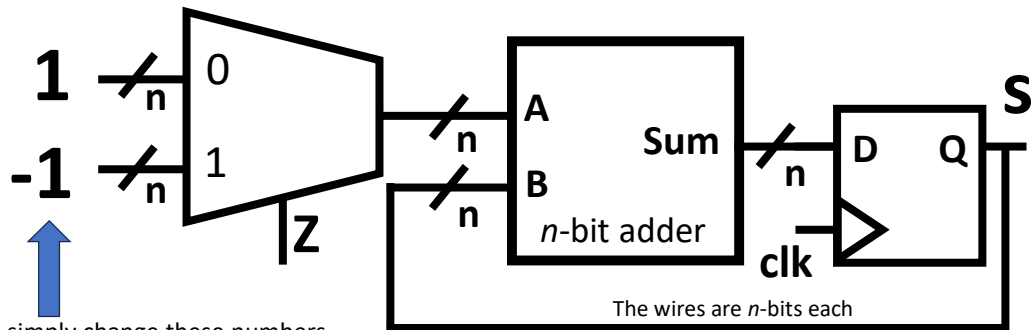
13

Now, let's try to complicate things a bit. Let's say we want the accumulator to also support counting up (which is what we have already done) and counting down (performing -1 instead of +1). Note that we are adding a new input here, Z. Z will determine whether we have to count up or count down. As you can see from the problem statement, we have an if statement selecting what happens if Z is 0 or if Z is 1. Z selects whether we should add a 1 or a -1. One of the logic blocks that we had described last meeting can be used to implement this functionality!



Introducing more complexity

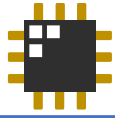
- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)



You can simply change these numbers to make it increment with any value!

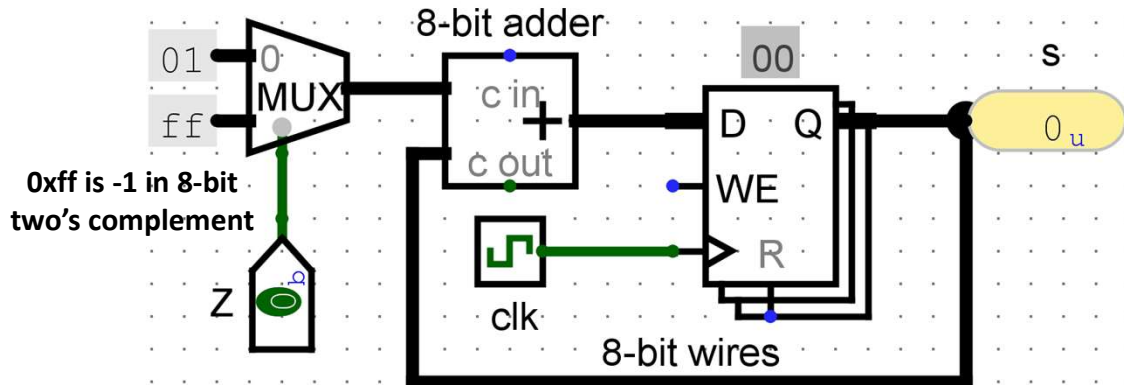
14

Using a multiplexer, we can select whether we add a 1 or a -1 to the adder. As you can see from the diagram, **Z** is the selector of the multiplexer, selecting whether to get +1 (if **Z = 0**) or -1 (if **Z = 1**). Thus, we can control what constant goes to the adder to be added to **s**. If **Z = 0**, the adder performs $s + 1$. If **Z = 1**, the adder performs $s + -1$ or $s - 1$. This is exactly the functionality that we desire!



Introducing more complexity

- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)



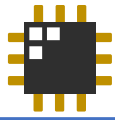
Here's a corresponding demonstration of the previous circuit. As you can see the multiplexer is selected from two constants, both are written in hex in this simulator. 0x01 corresponds to the constant 1 and 0xFF corresponds to the constant -1 (it is written in two's complement notation)

Let's say we have 8-bits:

1 = 0000 0001

-1 => 1111 1110 (invert everything) + 1 = 1111 1111 = 0xFF.

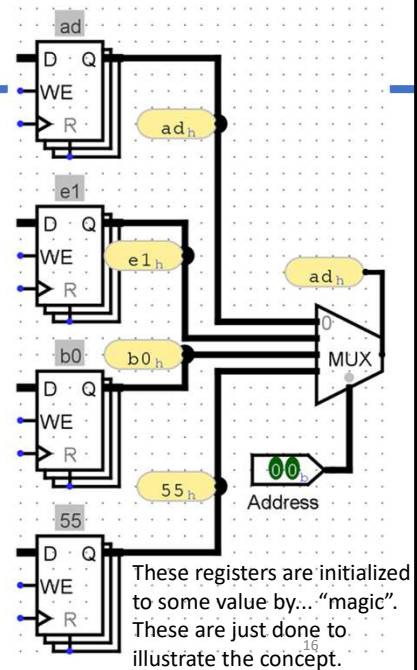
We can see that depending on the value of Z, the accumulator will count up or count down, exactly as what we wanted it to do.



Memory structures

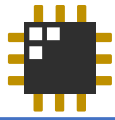
- Flipflops are memory elements – they hold data. You can then select from different flipflops using a multiplexer.
- You can think of memories like this
 - Flipflops are initialized to some specific value
 - All flipflop outputs go to a multiplexer for selection
 - The multiplexer selector becomes the **memory address**

Note: Actual read-only memories are not really made out of flipflops but this analogy is simple enough to imagine. Memory architectures are explored in EEE 153.



Flipflops are technically memory elements, they hold their value and will only change if there's a positive edge event. We can leverage this concept to implement more complex memory structures. As shown in the simulation on the right, we can utilize a multiplexer to select which of the flipflop outputs to be the final output. We already know how multiplexers work and what is being shown is typical multiplexer functionality. Since the multiplexer selector is technically selecting which flipflop (memory) output is used, we can imagine the selector as the 'address' of the memory. If the address is 00, it selects the first memory value; if 01, it selects the second memory value; and so on. You can also think of this as an array. Each array element will have their own values and each element is indexed. The index is the address in this illustration. When we provide the index of the array to get the value of that specific element, we are giving this read-only memory an address to get a value of that specific memory location.

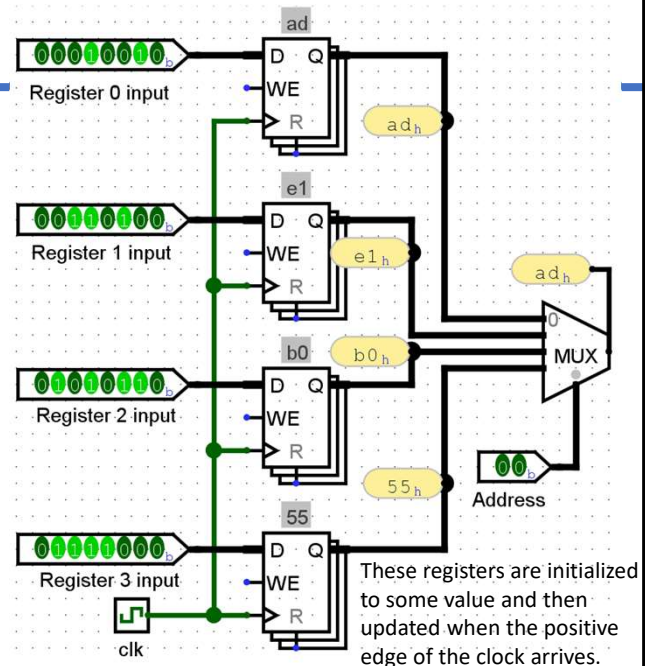
Note that this structure being demonstrated, a read-only memory, is just a conceptual illustration and does not reflect how real memories are implemented. Memory architectures are explored in EEE 153.



Memory structures

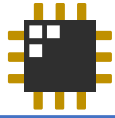
- If you want memories where you can **dynamically change contents**, you need to provide inputs to the flipflops. They also need a clock.
- This approach is... okay 
 - You need a lot of input though
 - Try to minimize the number of inputs by also **addressing** the writes

Note: Registers = flipflops with write enables
(you'll see the purpose of the write enables on the next slide!)



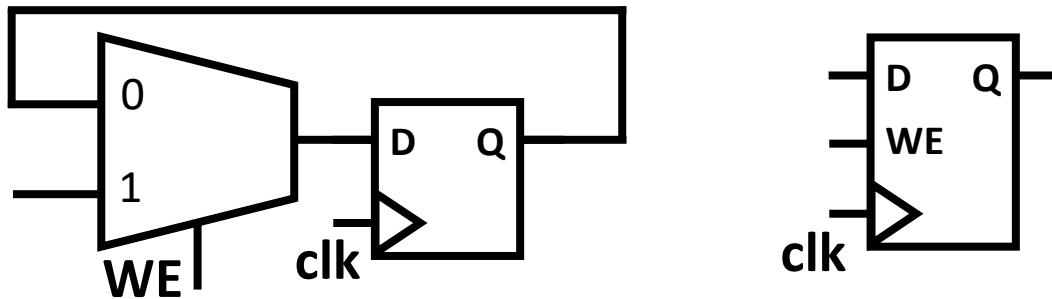
We can improve the memory structure by including a clock input and flipflop inputs. This allows us to dynamically change the contents of the memories (flipflops). Knowing that flipflops update on the positive edge of the clock, the flipflop inputs are copied over to the outputs on this positive edge trigger event, making it hold new memory values.

This approach, however, is still rather inefficient since implementing this memory would require a lot of inputs, since we require dedicated inputs per memory. If we have more memory elements, we'll end up with more inputs. A better solution is to dynamically address the memory writes, something like how we are addressing the reads through a multiplexer.



Memory structures

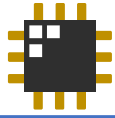
- Registers are flipflops with an additional input called **write enable (WE)**
 - If **WE = 1**, they behave similarly to a flipflop (update on positive edge).
 - If **WE = 0**, they just hold their values and ignore the clock completely.



If WE = 0, the multiplexer provides the same input to the flipflop, making it hold the current value even with a positive edge of the clock

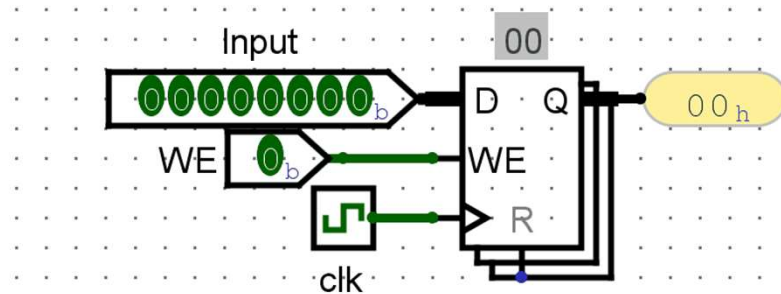
18

To implement a write-addressable memory. We now have to utilize the write enable (WE) inputs. The WE inputs control when the register (a register is just a flipflop with a WE input) behaves like a normal flipflop or if it ignores the positive edge of the clock completely (making it just hold the current value indefinitely). This WE input allows us to selectively update only one of the memory elements and not write to all the rest of the memory elements.



Memory structures

- Registers are flipflops with an additional input called **write enable (WE)**
 - If **WE = 1**, they behave similarly to a flipflop (update on positive edge).
 - If **WE = 0**, they just hold their values and ignore the clock completely.

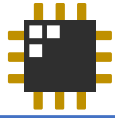


8-bit register

Note the behavior if WE = 0 and WE = 1.

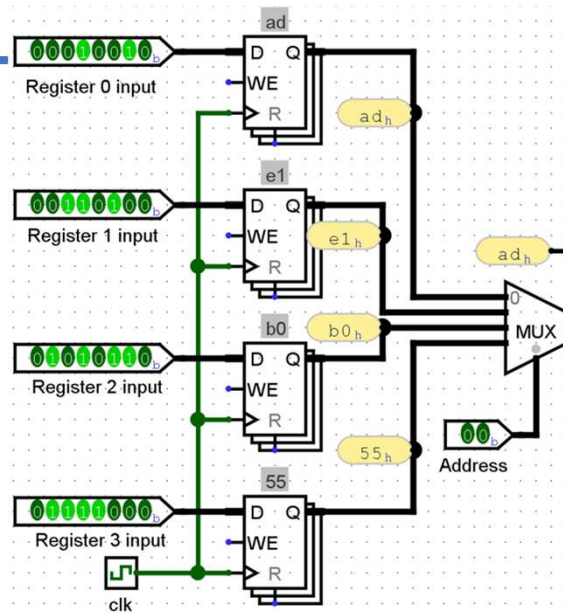
19

Here's a demonstration of the WE functionality of the register. It shows that if WE = 1, the register just functions like a typical flipflop, copying output every positive edge of the clock. However, if WE = 0, then the register will just hold the current value even if the inputs change and the positive edge of the clock arrives.

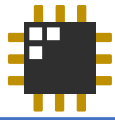


Memory structures

- This approach is... okay 
 - You need a lot of input though
 - Try to minimize the number of inputs by also **addressing** the writes
- Let's utilize **WE** to improve this.
 - Given an address and one 8-bit input, selectively write to only one of the four registers every positive edge of the clock.
 - We need one of the blocks introduced last meeting. *Hint: 1-to-many*

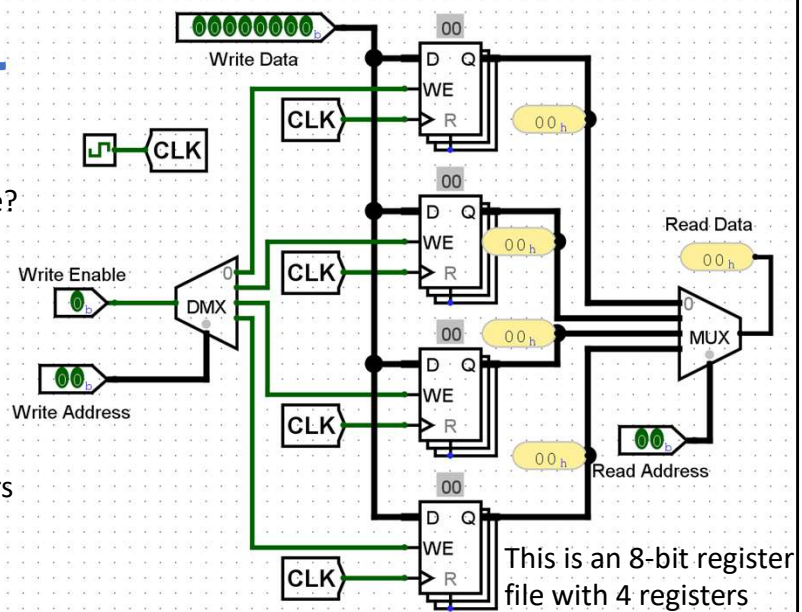


So going back to the previous memory structure that we had, we can now attempt to utilize the WE inputs of these registers so that we can dynamically update only one of the memories (registers) at any given time. We would need a logic block that has a 1-to-many functionality. It would be a demultiplexer!



Register file

- A register file is a collection of registers
 - **Write Address:** where to write?
 - **Write Data:** what to write?
 - **Write Enable:** should I write?
 - **Read Address:** where to read?
 - **Read Data:** what was read?
 - **CLK:** The trigger for all registers
- A register file is one main component of a processor!

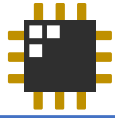


Using a demultiplexer, we will end up with the following structure on the right. The demultiplexer selects which of the many (in this case 4) memory elements will be updated through the pair of Write Enable input (which is the input being propagated to one of the demux outputs) and Write Address input (which is the selector to determine which way the input to propagated to).

We also have the Write Data which is actually connected to all the register inputs. This is the data that the specific register will update to, as long as the WE is 1 and there is a positive edge of the clock.

The multiplexer on the right simply selects which register output is being read. The Read Address is the selector and the Read Data is the output of the multiplexer, reflecting the memory output that was selected.

This entire structure is called a register file. This is actually one of the main components of a processor! We will be building a simple processor in the next lecture that why this concept has been introduced to you right now.



Looking forward

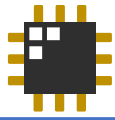
- Once we have the concept of memory to be included with logic gates, we can do more *fun* stuff
 - We can generate output *based on what happened before*.
 - We can design systems that generate a *sequence*.



GIFs from tenor.com

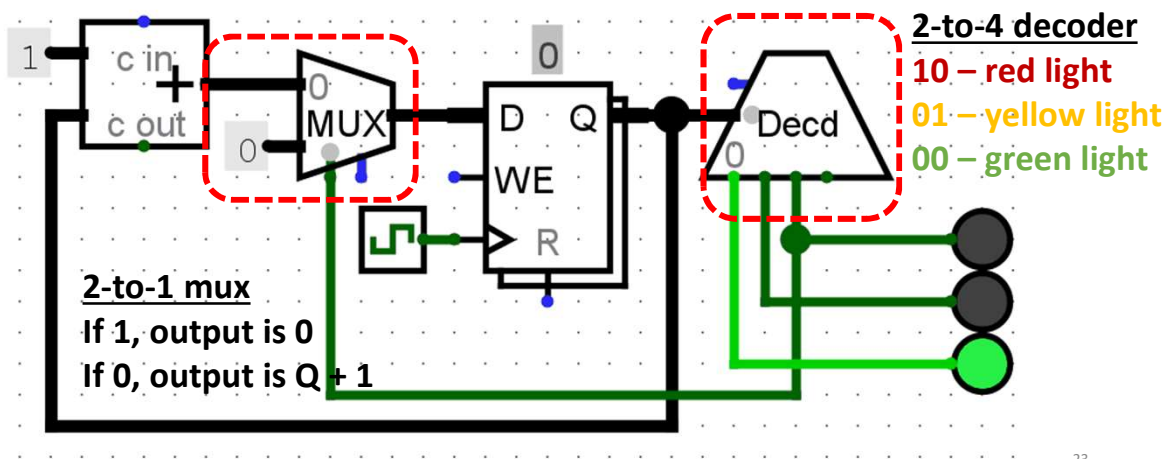
22

And so, with the concept of logic gates and memory elements like flipflops and registers, we can implement more complex (and fun) stuff that can generate output not just on the current input but also based on what happened before. We can also design systems that generate a specific sequence even without inputs. Imagine a traffic light that transitions to the different colors in sequence or a washing machine that cycles through the different wash and dry states in sequence.

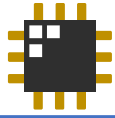


Just for fun

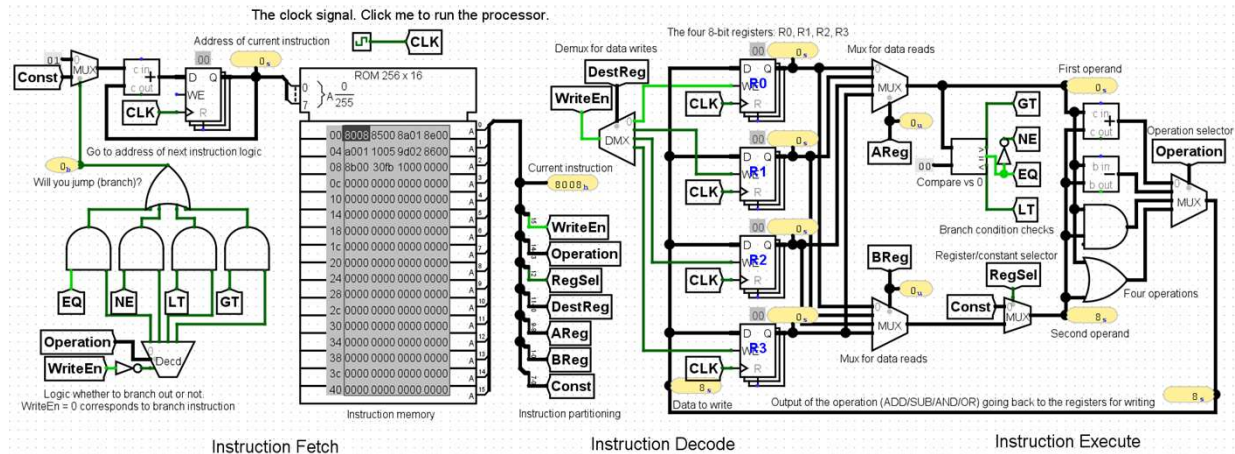
- Traffic light controller example. No inputs, just clock!



This is a simulation of the traffic light machine using a 2-bit register (since we need 3 different colors, we need at least 2 bits to represent each of them). The 2-to-4 decoder on the right would select which light to turn on given the current 2-bit value of the register. There's only three different colors so we only map 3 of the decoder outputs. The multiplexer on the left ensures that the register will not overflow. Since the adder always performs +1 on the current register value, there might be a case where we transition from 00 -> 01 -> 10 -> 11. The last 11 is problematic since we only have three lights and 11 does not map to any color. Thus, the multiplexer checks whether the current register value is now 10 (this is acquired from the decoder's input), and then selects 0 to be the next register value. Thus, we will end up with a 00 -> 01 -> 10 transition which loops back to 00.

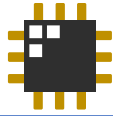


Next meeting spoilers: Your first processor

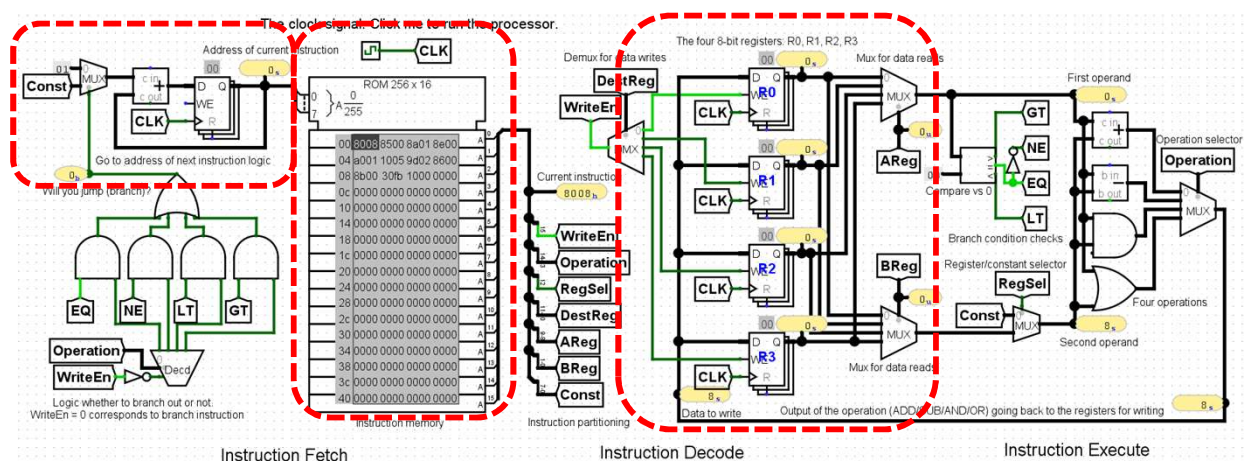


24

As described earlier, memories allow us to implement more complex logic, even a processor. Here's a sample processor that we will be building in our next lecture. It looks complex right now...



Next meeting spoilers: Your first processor



These blocks are variants of the ones we covered today!

25

However, looking closely, some of the block arrangements are variants of the blocks that we have covered in this lecture. The leftmost block, for instance, is like the up/down counter that we designed earlier. The structure is almost the same, except for a constant being added instead of -1. The middle block is a read-only memory. We have illustrated earlier how a read-only memory can be implemented: simply memory elements that are multiplexed, selected by an address. And finally, we have the block on the right is the register file that we have implemented towards the end of the lecture. It has 4 registers, with a demultiplexer to select which ones to write into, and a multiplexers to select which ones to read. This is almost exactly the same as the register file we designed, except for the fact that there's two multiplexers at the output to select two different register values at any given time. We will be building this processor slowly in our next lecture, so watch out how each of the blocks that we've designed would play a role in the development of this processor!